# FIND THAT BOOK

## Architecture & Implementation Blueprint

AI-Powered Library Discovery Workflow

.NET 8 Web API + Gemini AI + React Frontend + Open Library API

InfoTrack Technical Challenge

Author: **Oscar Alvarez**

Date: March 2026

Time Constraint: **2 Hours Implementation Sprint**

# TABLE OF CONTENTS

# 1. Executive Summary

**Mission Statement**

Build an AI-powered book discovery service that accepts messy, plain-text queries (author fragments, partial titles, character names, noisy tokens) and intelligently resolves them to canonical book records from Open Library. The system uses LLM-based field extraction as its core differentiator, backed by a Clean Architecture .NET 8 Web API and a modern React frontend.

| 2h | 5 | 3 | AI |
|---|---|---|---|
| Sprint Window | Core Layers | API Endpoints | Gemini-Powered |

This document provides the complete architecture, project structure, API contracts, AI integration strategy, and a minute-by-minute implementation plan optimized for a 2-hour delivery window. Every design decision is justified against the evaluation criteria: code quality, API design, AI integration, error handling, testing, and creative problem solving.

# 2. Requirements Analysis

## 2.1 Functional Requirements

| # | Requirement | Priority | Complexity |
|---|---|---|---|
| FR-01 | Accept messy plain-text queries (sparse, noisy, ambiguous) | MUST | Medium |
| FR-02 | AI-powered field extraction: title, author, keywords from blob | MUST | High |
| FR-03 | Search Open Library API (/search.json, /works, /authors) | MUST | Medium |
| FR-04 | Normalize inputs: lowercase, punctuation, diacritics, partials | MUST | Medium |
| FR-05 | Apply matching hierarchy (exact > near-match > author-only) | MUST | High |
| FR-06 | De-duplicate to canonical works, resolve primary authors | MUST | Medium |
| FR-07 | Return ordered candidates with explanation per match | MUST | Low |

| | | | |
|---|---|---|---|
| FR -08 | Optional AI re-ranking of top candidates | SHOULD | Medium |
| FR -09 | Modern UI to display results with cover images | MUST | Medium |
| FR -10 | Handle contributors vs. primary authors distinction | MUST | Medium |

## 2.2 Non-Functional Requirements

| Category | Requirement | Target |
|---|---|---|
| Performance | End-to-end response time | < 3 seconds (warm) |
| Resilience | Open Library API timeout handling | 3s timeout + retry (1x) |
| AI Failover | Graceful degradation if Gemini is unavailable | Regex-based fallback extraction |
| Testability | Unit test coverage on core logic | > 80% on Domain + Application |
| API Design | RESTful, versioned, documented | OpenAPI/Swagger + examples |
| Security | API key not exposed to frontend | Server-side only |
| Scalability | Stateless API ready for horizontal scale | No server-side session |

## 2.3 Evaluation Criteria Mapping

> **Strategy: Every architectural decision maps to an evaluation criterion**
>
> Code Quality -> Clean Architecture with 5 layers | API Design -> RESTful with versioning + Swagger | AI Integration -> Gemini with structured prompts + fallback | Error Handling -> Global middleware + Result pattern | Testing -> xUnit + Moq at unit/integration level | Communication -> This architecture doc + inline code comments | Creative Problem Solving -> AI re-ranking + fuzzy matching + multi-strategy search

# 3. System Architecture

## 3.1 Architecture Style: Clean Architecture

The system follows Clean Architecture (Robert C. Martin) with dependency inversion. The domain layer has zero external dependencies. All I/O flows through interfaces defined in the Application layer, implemented in Infrastructure. This ensures testability, maintainability, and clear separation of concerns.

## 3.2 C4 Level 1: System Context

The system context shows three external actors and two external systems. The User interacts through a React SPA which calls the .NET 8 API. The API orchestrates between Gemini AI (for NLP extraction) and Open Library (for book data).

```
C4 CONTEXT DIAGRAM

[User / Browser]
        |
        v  HTTP/JSON
[React SPA]  ------->  [.NET 8 Web API]
        |                  |
        v                  v
[Gemini AI API]    [Open Library API]
  (Field Extraction)   (Book Data Source)
```

## 3.3 C4 Level 2: Container View - Clean Architecture Layers

Five distinct layers ensure separation of concerns. Dependencies flow inward: API -> Application -> Domain. Infrastructure implements interfaces defined by Application.

| Layer | Project Name | Responsibility | Key Dependencies |
|---|---|---|---|
| Presentation | FindThatBook.Api | Controllers, middleware, DI config, Swagger | Microsoft.AspNetCore |
| Application | FindThatBook.Application | Use cases, DTOs, interfaces, orchestration | MediatR (optional) |
| Domain | FindThatBook.Domain | Entities, value objects, matching logic, enums | None (zero dependencies) |
| Infrastructure | FindThatBook.Infrastructure | Gemini client, Open Library client, HTTP | Google.AI, HttpClient |

| | | | |
|---|---|---|---|
| Tests | FindThatBook.Tests | Unit + Integration tests | xUnit, Moq, FluentAssertions |

**CLEAN ARCHITECTURE LAYERS (Dependency Flow)**

**[ API Layer - Controllers, Middleware, Swagger ]**

| depends on |

**[ Application Layer - Use Cases, DTOs, Interfaces ]**

| depends on |

**[ Domain Layer - Entities, Value Objects, Logic ]**

^ implements ^

**[ Infrastructure Layer - Gemini Client, OpenLibrary Client, HTTP ]**

# 4. AI Integration Strategy

## 4.1 Gemini AI: Field Extraction Pipeline

The AI layer is the core differentiator. Instead of simple regex parsing, we use Google Gemini to understand messy input semantically. The LLM receives the raw query blob and produces structured JSON with extracted fields.

> **AI Prompt Engineering Strategy**
>
> System Prompt: "You are a librarian assistant. Given a messy text blob that may contain book titles, author names, character names, years, and noise, extract structured fields. Return ONLY valid JSON." The prompt includes few-shot examples: "mark huckleberry" -> {title: "Adventures of Huckleberry Finn", author: "Mark Twain", keywords: ["huckleberry"]} to teach the model about partial name resolution and character-to-book mapping.

## 4.2 AI Response Schema

| Field | Type | Description | Example |
|---|---|---|---|
| title | string \| null | Extracted or inferred book title | "The Hobbit" |
| author | string \| null | Extracted or inferred author name | "J.R.R. Tolkien" |
| keywords | string[] | Additional tokens for search | ["illustrated", "deluxe"] |
| year | int \| null | Publication year if mentioned | 1937 |
| confidence | string | AI confidence: high, medium, low | "high" |
| reasoning | string | Why these fields were extracted | "Recognized 'tolkien hobbit'..." |

## 4.3 Fallback Strategy: Graceful Degradation

If Gemini is unavailable (rate limit, timeout, error), the system falls back to a regex-based extractor that uses heuristics: capitalized words as potential titles, known author patterns, year detection via 4-digit numbers, and stop-word removal. This ensures the service never fails completely.

> **AI FALLBACK CHAIN**
>
> 1. Try Gemini AI extraction (timeout: 5s)
> 2. On failure -> Regex-based field extraction (< 10ms)
> 3. On partial extraction -> Merge AI + Regex results

> 4. Always proceed to Open Library search with best-effort fields

## 4.4 Optional AI Re-Ranking

After Open Library returns candidates, we optionally send the top 10 results back to Gemini with the original query for intelligent re-ranking. The AI evaluates semantic similarity between the query intent and each candidate, producing a final ordered list with explanations. This is marked OPTIONAL due to the 2-hour constraint and additional API latency.

# 5. API Design

## 5.1 RESTful Endpoints

| Method | Endpoint | Description | Response |
|---|---|---|---|
| POST | /api/v1/books/search | Main search: accepts query blob, returns ranked results | BookSearchRespons e |
| GET | /api/v1/books/{workId} | Get full details for a specific Open Library work | BookDetailResponse |
| GET | /api/v1/health | Health check including AI + Open Library connectivity | HealthResponse |

## 5.2 Request/Response Contracts

### POST /api/v1/books/search - Request

**BookSearchRequest**

{ "query": "tolkien hobbit illustrated deluxe 1937", "maxResults": 5, "enableAiReranking": true }

### POST /api/v1/books/search - Response

**BookSearchResponse**

{ "query": "tolkien hobbit illustrated deluxe 1937", "extraction": { "title": "The Hobbit", "author": "J.R.R. Tolkien", "keywords": ["illustrated", "deluxe"], "year": 1937, "confidence": "high", "method": "gemini" }, "results": [ { "title": "The Hobbit", "author": "J.R.R. Tolkien", "firstPublishYear": 1937, "openLibraryWorkId": "/works/OL45804W", "openLibraryUrl": "https://openlibrary.org/works/OL45804W", "coverImageUrl": "https://covers.openlibrary.org/b/id/12345-M.jpg", "matchRank": 1, "matchType": "exact_title_primary_author", "explanation": "Exact title match; Tolkien is primary author; keywords 'illustrated deluxe' match edition variants." } ], "totalCandidates": 12, "processingTimeMs": 1840 }

## 5.3 Error Handling Strategy

The API uses a Result<T> pattern internally (no exceptions for business logic). A global exception middleware catches unhandled errors and returns consistent ProblemDetails (RFC 7807) responses. All external calls (Gemini, Open Library) are wrapped in try-catch with specific error types.

| HTTP Code | Scenario | Response Body |
|---|---|---|
| 200 | Successful search with results | BookSearchResponse |
| 200 | No results found (not an error) | BookSearchResponse with empty results |
| 400 | Missing or empty query field | ProblemDetails with validation errors |
| 408 | Open Library timeout (all retries exhausted) | ProblemDetails |
| 429 | Rate limit from Gemini (falls back to regex) | Still returns 200 with fallback |
| 500 | Unexpected server error | ProblemDetails with correlation ID |

# 6. Domain Model & Matching Logic

## 6.1 Core Entities

| Entity | Properties | Purpose |
|---|---|---|
| SearchQuery | RawText, ExtractedTitle, ExtractedAuthor, Keywords[], Year | Encapsulates the parsed query |
| BookCandidate | Title, Authors[], FirstPublishYear, WorkId, CoverIds[] | Raw result from Open Library |
| MatchedBook | BookCandidate + MatchRank, MatchType, Explanation | Ranked result with explanation |
| ExtractionResult | Title, Author, Keywords[], Year, Confidence, Method | AI extraction output |
| AuthorInfo | Name, Key, Role (primary/contributor/editor/illustrator) | Distinguishes author types |

## 6.2 Matching Hierarchy (Domain Logic)

The matching algorithm implements a 5-tier hierarchy as specified in the requirements. Each tier produces a different MatchType enum value and explanation template. The algorithm processes all candidates through all tiers and assigns the highest applicable tier to each.

| Tier | Match Type | Logic | Score Weight |
|---|---|---|---|
| 1 (Strongest) | ExactTitlePrimaryAuthor | Normalized title == extracted title AND primary author matches | 100 |
| 2 | ExactTitleContributorAuthor | Normalized title matches BUT author is contributor only | 80 |
| 3 | NearMatchTitleAuthor | Fuzzy title match (Levenshtein < 3) + author match | 60 |
| 4 | AuthorOnlyFallback | No title match but author matches -> return top works | 40 |
| 5 (Weakest) | KeywordCandidate | Multiple keyword overlaps without strong title/author match | 20 |

## 6.3 Normalization Pipeline

> **Input Normalization Steps (Applied Before Matching)**

> 1. Lowercase all text -> 2. Remove diacritics (Tolkien/tolkien) -> 3. Strip punctuation except hyphens -> 4. Collapse whitespace -> 5. Remove common noise words (the, a, an, of, and, by) -> 6. Handle subtitle variants ("The Hobbit" == "The Hobbit, or There and Back Again") -> 7. Trim trailing/leading whitespace

# 7. Project Structure

## 7.1 Solution Layout

```
FindThatBook/
  FindThatBook.sln
  src/
    FindThatBook.Api/
      Controllers/BookSearchController.cs
      Middleware/GlobalExceptionMiddleware.cs
      Program.cs, appsettings.json
    FindThatBook.Application/
      UseCases/SearchBooksUseCase.cs
      DTOs/BookSearchRequest.cs, BookSearchResponse.cs
      Interfaces/IAiExtractor.cs, IBookRepository.cs
      Mapping/BookMapper.cs
    FindThatBook.Domain/
      Entities/SearchQuery.cs, BookCandidate.cs, MatchedBook.cs
      ValueObjects/ExtractionResult.cs, AuthorInfo.cs
      Enums/MatchType.cs, ExtractionMethod.cs
      Services/BookMatchingService.cs, TextNormalizer.cs
    FindThatBook.Infrastructure/
      AI/GeminiExtractor.cs, RegexFallbackExtractor.cs
      OpenLibrary/OpenLibraryClient.cs, OpenLibraryModels.cs
      DependencyInjection/ServiceCollectionExtensions.cs
  tests/
    FindThatBook.Tests/
      Unit/BookMatchingServiceTests.cs
      Unit/TextNormalizerTests.cs
      Unit/GeminiExtractorTests.cs
      Integration/SearchBooksUseCaseTests.cs
      Integration/OpenLibraryClientTests.cs
  client/  (React frontend)
    src/App.tsx, components/, hooks/, types/
  docker-compose.yml, README.md, .github/workflows/ci.yml
```

## 7.2 Key Design Patterns

| Pattern | Where Applied | Benefit |
|---|---|---|
| Strategy Pattern | IAiExtractor: Gemini vs. Regex | Swap AI providers without changing logic |
| Result<T> Pattern | All use cases return Result<T> | No exceptions for business logic flow |
| Repository Pattern | IBookRepository for Open Library | Abstracts external API behind interface |

| | | |
|---|---|---|
| Dependency Injection | All layers use constructor injection | Testable, loosely coupled components |
| Middleware Pipeline | Global exception + logging + CORS | Cross-cutting concerns handled once |
| Options Pattern | GeminiOptions, OpenLibraryOptions | Typed configuration with validation |

# 8. Search Flow: End-to-End Sequence

This section traces a complete search request from the user typing a query to seeing results on screen. Every step is annotated with the responsible layer and estimated time.

| Step | Action | Layer | Time Est. |
|---|---|---|---|
| 1 | User types "mark huckleberry" and clicks Search | Frontend | 0ms |
| 2 | React sends POST /api/v1/books/search with query | Frontend | ~10ms |
| 3 | Controller validates request, calls SearchBooksUseCase | API | ~1ms |
| 4 | Use case sends raw query to IAiExtractor (Gemini) | Application | ~800ms |
| 5 | Gemini returns: {author: "Mark Twain", title: "Huckleberry Finn"} | Infrastructure | included |
| 6 | Use case builds multi-strategy Open Library queries | Application | ~1ms |
| 7 | Query 1: search.json?title=huckleberry+finn&author=mark+twain | Infrastructure | ~400ms |
| 8 | Query 2: search.json?q=huckleberry+finn (broad fallback) | Infrastructure | ~400ms |
| 9 | De-duplicate results by work_id, fetch /works/{id} for details | Infrastructure | ~300ms |
| 10 | BookMatchingService scores and ranks all candidates | Domain | ~5ms |
| 11 | Generate explanation per candidate based on MatchType | Domain | ~1ms |
| 12 | (Optional) Send top 10 to Gemini for AI re-ranking | Infrastructure | ~600ms |
| 13 | Return ordered BookSearchResponse to controller | Application | ~1ms |
| 14 | Controller returns JSON response with 200 OK | API | ~1ms |
| 15 | React renders result cards with covers and explanations | Frontend | ~50ms |

**Total Estimated Response Time**

Without AI Re-Ranking: ~1.5 - 2.0 seconds | With AI Re-Ranking: ~2.0 - 2.6 seconds | These estimates assume warm connections and no retries. First request may be slower due to connection establishment.

# 9. Testing Strategy

## 9.1 Test Pyramid

| 15+ | 5+ | 3+ |
|---|---|---|
| Unit Tests | Integration Tests | Edge Case Tests |

| Level | Target | Example Tests | Framework |
|---|---|---|---|
| Unit | TextNormalizer | Diacritics removal, subtitle handling, whitespace collapse | xUnit |
| Unit | BookMatchingService | Tier 1-5 scoring, de-duplication, author role resolution | xUnit + Moq |
| Unit | GeminiExtractor | JSON parsing, malformed response handling, timeout | xUnit + Moq |
| Unit | RegexFallbackExtractor | Year detection, title/author heuristics, edge cases | xUnit |
| Integration | SearchBooksUseCase | Full pipeline with mocked Gemini + real Open Library | xUnit + WebApplicationFactory |
| Integration | OpenLibraryClient | Real API calls with various query types | xUnit (conditional) |
| Edge Case | Empty/null queries | Whitespace-only, single character, special chars | xUnit |
| Edge Case | Non-English queries | Accented characters, CJK, mixed scripts | xUnit |
| Edge Case | API failures | Gemini timeout + OL timeout simultaneously | xUnit + Moq |

## 9.2 Key Test Scenarios from Requirements

**Must-Pass Test Cases (from Assessment Examples)**

1. "dickens, tale two cities" -> Should find A Tale of Two Cities by Charles Dickens2. "mark huckleberry" -> Should find Adventures of Huckleberry Finn by Mark Twain (partial/character name)3. "twilight meyer" -> Should find Twilight by Stephenie Meyer4. "tolkien hobbit illustrated deluxe 1937" -> Should find The Hobbit with noise filtering5. "austen bennet" -> Should find Pride and Prejudice (character name to book mapping)

# 10. Frontend Architecture

## 10.1 Technology Stack

| Technology | Purpose | Justification |
|---|---|---|
| React 18 | UI framework | Component-based, excellent ecosystem, fast rendering |
| TypeScript | Type safety | Catches errors at compile time, better DX |
| Tailwind CSS | Styling | Rapid prototyping, consistent design, no CSS files |
| Vite | Build tool | Fast dev server, instant HMR, optimized production builds |
| React Query | Data fetching | Caching, loading states, error handling built-in |

## 10.2 UI Components

| Component | Description | Key Features |
|---|---|---|
| SearchBar | Main input with search button | Debounced input, loading spinner, example queries |
| ResultCard | Individual book result card | Cover image, title, author, year, match explanation |
| ExtractionPanel | Shows AI extraction details | Extracted fields, confidence badge, method indicator |
| EmptyState | No results messaging | Helpful suggestions for refining the query |
| ErrorBoundary | Global error handling | Graceful fallback UI with retry option |
| LoadingSkeleton | Placeholder during fetch | Animated skeleton cards matching result layout |

# 11. Implementation Plan: 2-Hour Sprint

**Sprint Strategy: Ship the Core, Polish What Matters**

The 2-hour window requires ruthless prioritization. We focus on: (1) working end-to-end flow, (2) clean code that demonstrates architecture, (3) AI integration that actually works, (4) enough tests to prove quality. Items marked STRETCH are attempted only if time permits.

| Time | Duration | Task | Priority |
|---|---|---|---|
| 0:00 - 0:10 | 10 min | Project scaffolding: dotnet new, solution structure, NuGet packages | MUST |
| 0:10 - 0:20 | 10 min | Domain layer: entities, value objects, enums, TextNormalizer | MUST |
| 0:20 - 0:35 | 15 min | Domain: BookMatchingService with 5-tier hierarchy | MUST |
| 0:35 - 0:50 | 15 min | Infrastructure: GeminiExtractor with prompt + JSON parsing | MUST |
| 0:50 - 1:00 | 10 min | Infrastructure: OpenLibraryClient with search + works endpoints | MUST |
| 1:00 - 1:15 | 15 min | Application: SearchBooksUseCase orchestrating the full pipeline | MUST |
| 1:15 - 1:25 | 10 min | API: Controller + middleware + Swagger + DI registration | MUST |
| 1:25 - 1:35 | 10 min | Infrastructure: RegexFallbackExtractor as AI failover | MUST |
| 1:35 - 1:45 | 10 min | Tests: Core unit tests for matching + normalization | MUST |
| 1:45 - 1:55 | 10 min | Frontend: React SPA with search bar + result cards | MUST |
| 1:55 - 2:00 | 5 min | README, final testing, cleanup, commit + push | MUST |

## 11.1 Stretch Goals (If Time Permits)

| Goal | Value | Estimate |
|---|---|---|
| AI Re-Ranking endpoint | Demonstrates advanced AI usage | +15 min |
| Docker Compose setup | Easy deployment for evaluators | +10 min |
| Integration tests with WebApplicationFactory | Proves full pipeline works | +10 min |
| GitHub Actions CI/CD pipeline | Shows DevOps awareness | +5 min |
| Live deployment to Azure/Railway | Immediate demo access | +10 min |

# 12. Key Technical Decisions

## 12.1 Decision Log (ADRs)

| Decision | Choice | Rationale | Alternatives Considered |
|---|---|---|---|
| AI Provider | Google Gemini (free tier) | Per assessment requirements; free API key; strong JSON mode | OpenAI GPT-4, Claude, local Ollama |
| Architecture | Clean Architecture (5 layers) | Evaluation criteria: code quality + organization | Vertical slices, simple 3-layer |
| Frontend | React + TypeScript + Tailwind | Modern, fast to prototype, shows full-stack ability | Blazor, Angular, vanilla JS |
| HTTP Client | Typed HttpClient with Polly | Resilience policies: retry, timeout, circuit breaker | Raw HttpClient, RestSharp |
| Testing | xUnit + Moq + FluentAssertions | .NET standard, expressive assertions, easy mocking | NUnit, MSTest |
| Error Handling | Result<T> + ProblemDetails | No exception-driven flow; RFC 7807 standard errors | Exceptions everywhere, custom format |
| Matching | Levenshtein + custom normalization | Handles typos and variants without external libs | Lucene.NET, FuzzySharp, ElasticSearch |

## 13. Future Improvements

With more time, these enhancements would elevate the solution from a technical challenge submission to a production-ready product:

| Category | Improvement | Impact |
|---|---|---|
| Performance | Redis cache for Open Library responses (TTL: 1h) | 50%+ faster repeat queries |
| Performance | Parallel Open Library API calls with Task.WhenAll | 30% faster search |
| AI | Fine-tuned prompt variants for different query types | Better extraction accuracy |
| AI | Embedding-based similarity search for re-ranking | More nuanced matching |
| UX | Search history + autocomplete suggestions | Faster repeat searches |
| UX | Book preview with synopsis, reviews, related works | Richer discovery experience |
| Infrastructure | Rate limiting + API key management | Production-ready security |
| Infrastructure | Structured logging with Serilog + Seq | Observability and debugging |
| Testing | Property-based testing with FsCheck | Edge case coverage |
| Testing | Contract tests against Open Library API | Detect API changes early |

## 14. Summary

**Ready to Execute**

This architecture document provides a complete, actionable blueprint for the Find That Book challenge. Every layer, interface, and endpoint is specified. The 2-hour sprint plan has been designed to deliver a working, well-architected solution that demonstrates: Clean Architecture with proper separation of concerns, real AI integration with Gemini (not just a wrapper), creative problem solving in the matching and normalization pipeline, comprehensive error handling and fallback strategies, and production-quality code organization. The implementation begins now.

**Oscar Alvarez** | Senior Solutions Architect

23+ years of enterprise architecture experience

InfoTrack Technical Challenge - March 2026